CS 2001 Data Structures

Assignment 2

Spring 2026

DS and AI (all sections)

Instructions

1. **Guidelines Compliance:** Any violation of the guidelines provided will result in a **score of zero** for the assignment.
2. **Softcopy Submission:** The assignment must be submitted in soft form on the **GCR**.
3. **No Email Submissions:** Under **NO circumstances** will submissions via email or any other way be accepted.
4. **Submission Deadline:** 8 March 2026 11:59 PM
5. **Individual Work:** This is an **individual assignment**. Collaboration or copying will not be tolerated.
6. **Plagiarism: STRICT POLICY ON CHEATING** THROUGH ANY SOURCE. **ZERO MARKS** WILL BE ASSIGNED IN ALL ASSIGNMENTS AND QUIZZES.
7. **Identification Details:** Ensure that your **Roll Number, Section, Name, and program** are written in the code files. Rename the folder as **ROLL-NUM_PROGRAM_SECTION** (e.g., i240001_DS/AI_A) and compress the folder as a **zip file** (e.g., i240001_DS/AI_A.zip). **Strictly follow** this naming convention, otherwise marks will be deducted.
8. **Reference Material:** While you may consult textbooks or other resources for your understanding, ensure **all work is done in your own words/code**. Provide references at the end if you have used anything from anywhere. **AI (GPT) is not allowed at all.**
9. **Programming Language:** The programming language allowed for this assignment is **C++ only**.
10. **Good Programming Practices:** Add **comments** in code for good programming practices. Make sure that code is **neatly formatted and indented**. Some marks are dedicated to this.
11. **Completeness and Correctness of Code:** Code should be complete and correct. If there is a **syntax error** in the code, **zero marks** will be awarded in that part of the assignment.

12. **Submission Guidelines:**
    - For each question in your assignment, make a **separate .cpp/.h file** (e.g., for question 1, make q1.cpp, q1.h and so on).
    - Each file that you submit must contain your **name, student-id, and assignment number** on top of the file in the comments (**2 marks** will be allocated for this).
    - Combine all your work in **one folder**. The folder must contain **only .cpp files** (no binaries, no .exe files, etc.).
    - The student is **solely responsible** for checking the final file for issues like corrupt files, viruses, or mistakenly sent .exe files. If the teacher is not able to download the assignment from GCR, it will lead to **zero marks**. Your teachers should be able to run your code on their laptop later to verify your outputs.

Notes

- All the Outputs shown in the assignment are **just for your understanding**. There is no Hard and Fast rule to implement them exactly as is. **More Creativity and effort mean More Marks.**
- **Start early** so that you can finish it on time.
- Please ensure that **only concepts covered in class** are used for the assignment.

Question 1:

Emergency Response Coordination System (ERCS)

Background: A smart city has deployed an Emergency Response Coordination System (ERCS) to manage incoming emergency incidents such as accidents, fires, and medical cases. Due to memory and infrastructure constraints, the system must be implemented using **custom Stacks and Queues only**, without using STL containers, arrays for temporary storage, or Linked Lists (Singly or Doubly).

Each emergency case must be represented as a node containing the following fields:

- int caseID;
- string callerName;
- int severityLevel; // 1 (Low) → 10 (Critical)
- bool isResolved;
- Incident* next; // Used strictly for the underlying Stack/Queue implementation

System Requirements

1. Register a New Emergency Case

- When a citizen calls, a new emergency case must be enqueued into the main ActiveResponseQueue.
- **Constraints:** New cases are added chronologically (FIFO), memory must be dynamically allocated, and no arrays or STLs are allowed.

2. Dispatch and Close a Resolved Case

- When a case is handled, it must be dequeued from the front of the ActiveResponseQueue.
- **Constraints:** Memory must be deallocated properly with no memory leaks allowed.

3. Crisis Escalation Protocol

- During large-scale disasters, emergency cases are handled in priority waves of K incidents at a time.
- The system must use a temporary **Stack** to reverse the order of cases in the queue in groups of size K.

- **Constraints:** The last incomplete wave must remain unchanged.

4. Symmetry Audit

- At the end of each shift, authorities check whether the severity pattern of incidents currently in the queue forms a symmetric pattern.
    - *Example: 2 → 5 → 8 → 5 → 2.*
- The system must use a **Stack** to verify if the queue's severity levels read the same forwards and backwards.
- **Constraints:** The original queue must be perfectly restored after checking.

5. Risk-Based Segregation Protocol

- During extreme overload, cases must be reorganized from the main queue into two separate processing queues:
    - All cases with severity < X are dequeued and sent to a JuniorUnitsQueue.
    - All cases with severity ≥ X are dequeued and sent to a SpecialForcesQueue.
- **Constraints:** Preserve the original relative order of the cases when moving them.

Global Constraints

- Only custom Stack and Queue data structures are allowed.
- No STL containers are permitted.
- No arrays for temporary storage.
- No global variables or memory leaks.
- A menu-driven interface is required.

- - - - - - - -

Question 2 :

Deterministic Financial Transaction Orchestration Engine

Background:
A high-frequency digital payment platform processes live financial transactions across distributed gateways. The engine must preserve arrival fairness, detect fraud patterns, support reversible execution, enforce priority overrides, and guarantee deterministic replay under partial rollback. The entire system must be implemented **using only custom Stack and Queue** data structures without using STL, built-in containers, recursion, or global buffers.

Transaction Model
Each transaction contains the following attributes:
transactionID
timestamp
amount
riskScore
feeExpression (infix mathematical string)
dependencyTag (nested bracket structure)

System Requirements

1. Admission Layer
All incoming transactions must be preserved in strict arrival order. Direct queue implementation is not allowed. FIFO behavior must be simulated using two stacks only. Students must ensure and justify that no transaction reordering occurs.

2. Structural Validation
Each transaction includes a dependencyTag consisting of nested brackets. Before execution, the system must validate structural correctness using stack logic only. Malformed transactions must be rejected without disturbing the original arrival ordering.

3. Deferred Suspicion Holding
If a transaction's riskScore exceeds a threshold R, it must move to a Suspicion Holding Structure. This structure must behave as a circular queue. When system risk drops below threshold, held transactions must re-enter processing while preserving original relative order. No duplication is allowed.

4. Deterministic Fee Evaluation
Transaction fees are provided as infix mathematical expressions involving +, -, *, /, and parentheses. Students must convert infix expressions to postfix form and evaluate them using stack logic. Operator precedence must be handled manually. Divide-by-zero must be detected before state mutation.

5. Execution and State Mutation
A global variable globalBalance represents system state. Upon execution:
globalBalance = globalBalance + amount − evaluatedFee
Every successful execution must be recorded in an Execution Stack to support rollback.

6. Conditional Reverse Compensation
If globalBalance becomes negative, all transactions executed after the most recent high-value transaction must be undone in strict reverse order. Undone transactions must re-enter the admission structure without violating FIFO ordering.

7. Priority Escalation Window
Upon receiving ESCALATE X, all transactions with amount greater than or equal to X must execute before lower-amount transactions. Relative order within both groups must remain stable. No new array buffers may be used.

8. Sliding Fraud Analytics
For every window of size K over executed transactions, compute maximum amount, minimum amount, and maximum difference within the window. The solution must run in O(N) using manually implemented deque logic.

9. Load Shedding Simulation
If pending transactions exceed capacity C, the oldest half of waiting transactions must be temporarily reversed before processing continues. This reversal must affect only the waiting structure and must run in O(N) time using stack and queue operations only.

10. Deterministic Replay Guarantee
After any number of rollbacks, escalations, holding transfers, or load shedding operations, replaying the entire original transaction stream with the same commands must produce identical globalBalance and execution history. Students must maintain structural invariants ensuring determinism.

Mandatory Structural Constraints
1. One queue must be implemented using two stacks.
2. One stack must be implemented using queues.
3. Circular queue implementation is required.
4. Manual deque logic implementation is required.
5. No STL containers allowed.
6. No recursion allowed.
7. No arrays storing full history.
8. All operations must justify time complexity formally.

- - - - - - - -